



PROGRAMMATION FONCTIONNELLE via le langage Haskell

A. OUALI

L2 au département Mathématique-Informatique
UPR des sciences
Université de Caen Normandie



ZF-expressions¹ et filtrage par motifs (pattern matching)



- D'après la théorie des ensembles de Zermelo Fränkel
- Nom donné par David Turner pour les listes en compréhension

Forme générale :

```
[f x | x <- xs]
```

Filtrage par motifs :

```
[f x y | (x,y) <- xs]
```

1. Théorie d'ensemble - Zermelo Fränkel

A. Ouali

Programmation fonctionnelle en Haskell

2 / 41

ZF-expressions : exemples



```
> [x | x <- [1..10]] ==> [1,2,3,4,5,6,7,8,9,10]
> [x | x <- [1..10], even x] ==> [2,4,6,8,10]
> [x * x | x <- [1..10], even x] ==> [4,16,36,64,100]

> [(x,y) | x <- [1..2], y <- [5..7]]
==> [(1,5),(1,6),(1,7), (2,5),(2,6),(2,7)]
```

ZF-expressions : exemples



► La liste des diviseurs de l'entier naturel n

```
listDivisors :: Int -> [Int]
listDivisors n = [i | i <- [1..n], (mod n i) == 0]

> listDivisors 12 ==> [1,2,3,4,6,12]
> listDivisors 13 ==> [1,13]
```

ZF-expressions : exemples



► La liste des diviseurs de l'entier naturel n

```
listDivisors :: Int -> [Int]
listDivisors n = [i | i <- [1..n], (mod n i) == 0]

> listDivisors 12 ==> [1,2,3,4,6,12]
> listDivisors 13 ==> [1,13]
```

► n est-il un nombre premier?

```
isPrime :: Int -> Bool
isPrime n = (listDivisors n) == [1,n]

> isPrime 12 ==> False
> isPrime 13 ==> True
```

ZF-expressions : exemples



► La liste des nombres premiers $\leq k$

```
primesLessThan :: Int -> [Int]
primesLessThan k = [n | n <- [1..k], isPrime n]

> primesLessThan 20 ==> [2,3,5,7,11,13,17,19]
> primesLessThan 40 ==> [2,3,5,7,11,13,17,19,23,29,31,37]
```

ZF-expressions : exemples



► La liste des nombres premiers $\leq k$

```
primesLessThan :: Int -> [Int]
primesLessThan k = [n | n <- [1..k], isPrime n]

> primesLessThan 20 ==> [2,3,5,7,11,13,17,19]
> primesLessThan 40 ==> [2,3,5,7,11,13,17,19,23,29,31,37]
```

► Combien sont-ils?

```
nbPrimesLessThan :: Int -> Int
nbPrimesLessThan k = length (primesLessThan k)

> nbPrimesLessThan 20 ==> 8
> nbPrimesLessThan 40 ==> 12
```

ZF-expressions : exemples filtrage par motifs



- Étant donné une liste contenant des tuples du type : [(facteur, exposant), (facteur, exposant), (facteur, exposant)...], retourner la liste qui contient le résultat.

A. Ouali

Programmation fonctionnelle en Haskell

3 / 41

A. Ouali

Programmation fonctionnelle en Haskell

4 / 41

A. Ouali

Programmation fonctionnelle en Haskell

4 / 41

A. Ouali

Programmation fonctionnelle en Haskell

5 / 41

A. Ouali

Programmation fonctionnelle en Haskell

5 / 41

A. Ouali

Programmation fonctionnelle en Haskell

6 / 41

ZF-expressions : exemples filtrage par motifs



- ▶ Étant donné une liste contenant des tuples du type : [(facteur, exposant), (facteur, exposant), (facteur, exposant)...], retourner la liste qui contient le résultat.

```
Prelude> [ (**) a b | (a,b) <- [(7,3), (6,2)]  
[343.0,36.0]
```

Fonctions anonymes (λ-abstraction)



- ▶ La notation $\lambda x.f(x)$ désigne la fonction : $x \rightarrow f(x)$
 - ▶ successeur sur les entiers : $\lambda x.(x + 1)$
 - ▶ multiplier un nombre par 2 : $\lambda x.(2 * x)$
 - ▶ être un entier strictement positif : $\lambda x.(x > 0)$

- ▶ La notation Haskell

```
\x -> x + 1  
\x -> x * 2  
\x -> x > 0
```

- ▶ Utilisation analogue aux fonctions nommées

```
> (\x -> x + 1) 2 ==> 3  
> (\x -> x * 2) (7+5) ==> 24  
> (\x -> x > 0) (head [1, 2, 7]) ==> True
```

Fonctions anonymes (λ-abstraction)



- ▶ Fonctions (anonymes) avec plusieurs paramètres

```
\x y -> [x, y]      liste avec 2 éléments  
\x y -> (x+y)/2     moyenne de 2 nombres  
\x y -> (x, y)      couple
```

- ▶ Utilisation

```
> (\x y -> [x, y]) 6 85 ==> [6,85]  
> (\x y -> [x, y]) 'a' 'b' ==> "ab"  
> (\x y -> (x+y)/2) 7 14 ==> 10.5  
> (\x y -> (x, y)) 7 12 ==> (7,12)  
> (\x y -> (x, y)) 'a' 27 ==> ('a',27)  
> (\x y -> (x, y)) "azerty" True ==> ("azerty",True)
```

Fonctions anonymes (λ-abstraction)



Ce qu'il faudrait noter :

- ▶ ($\rightarrow$) est associé à la définition d'une fonction anonyme et à la déclaration des types
- ▶ ($\leftarrow$) est utilisé pour définir une ZF-expression

```
Prelude> [ (\x y -> x ^ y) a b | (a,b) <- [(7,3), (6,2)]  
[343,36]
```

ZF-expressions exercices



- ▶ Définir une fonction `occurrencesPos` qui renvoie les positions de toutes les occurrences du deuxième argument dans le premier argument.
Utiliser une définition avec une liste en compréhension. On se sert de la fonction `zip` ou `myZip` pour la définition.

- ▶ Exemple d'application :

```
Prelude> occurrencePos "Programmation" 'a'  
[5,6]
```

Curryfication : fonction avec plusieurs arguments



En exploitant le fait que les fonctions peuvent renvoyer des fonctions comme résultat :

- ▶ Les fonctions avec plusieurs arguments peuvent également être passées à une autre fonction

Une signature qui déclare une fonction comme un type de retour :

```
add :: Int -> (Int -> Int)  
add x y = x + y
```

- ▶ Toutefois, la définition déclare une fonction prenant x et y qui retourne l'addition $x + y$
- ▶ Même résultat dans les deux situations

```
add :: Int -> (Int -> Int) <=> add' :: (Int, Int) -> Int
```

Curryfication : fonction avec plusieurs arguments



En exploitant le fait que les fonctions peuvent renvoyer des fonctions comme résultat :

- ▶ En math, une fonction n'accepte qu'un seul argument, les fonctions avec plusieurs arguments utilisent la même technique (fonctions qui retournent des fonctions...)

```
mult :: Int -> (Int -> (Int -> Int))  
mult x y z = x * y * z
```

- ▶ Ce type de fonctions est appelé fonctions Curry

Curryfication : fonction avec plusieurs arguments



- ▶ Pour éviter les parenthèses, ' $\rightarrow$ ' supposé s'associer à droite. :

```
Int -> Int -> Int -> Int  
Int -> (Int -> (Int -> Int))
```

- ▶ Par conséquent, l'application de fonction s'associe à gauche :

```
mult x y z  
((mult x) y) z
```

Curryfication : fonction avec plusieurs arguments



En exploitant le fait que les fonctions peuvent renvoyer des fonctions comme résultat :

- ▶ En math, une fonction n'accepte qu'un seul argument, les fonctions avec plusieurs arguments utilisent la même technique (fonctions qui retournent des fonctions...)

```
mult :: Int -> (Int -> (Int -> Int))  
mult x y z = x * y * z
```

- ▶ Ce type de fonctions est appelé fonctions Curry

Curryfication : fonction avec plusieurs arguments



- ▶ Pour éviter les parenthèses, ' $\rightarrow$ ' supposé s'associer à droite. :

```
Int -> Int -> Int -> Int  
Int -> (Int -> (Int -> Int))
```

- ▶ Par conséquent, l'application de fonction s'associe à gauche :

```
mult x y z  
((mult x) y) z
```

Curryfication



```
Prelude> min 4 7      => 4

Prelude> (min 4) 7    => 4

Prelude> :t (min 4 7)      => (min 4 7) :: Int

Prelude> :t (min 4)       => (min 4) :: Int -> Int

Prelude> :t min          => min :: Int -> Int -> Int

Associativité à droite

> :t min      => min :: Int -> (Int -> Int)
```

Curryfication



Opérateur (+)

- (+) opérateur binaire infixé de somme
- (+ 2) fonction qui ajoute 2 à un nombre
- (+ 2) et \x->x+2 sont deux fonctions équivalentes

```
Prelude> 2 + 7
9
Prelude> (+ 2) 7
9
Prelude>> (\x -> x+2) 7
9
```

- les différents types (parenthésage implicite à droite)

```
(+) :: Int -> (Int -> Int)
(+ 2) :: Int -> Int
(+ 2) 7 :: Int
```

Curryfication



Opérateur (==)

- (==) opérateur binaire infixé
- (== 'a') fonction qui détermine si un caractère vaut 'a'
- (== 'a') et \x->x=='a' sont deux fonctions équivalentes

```
Prelude> 'b' == 'a'
False
Prelude> (== 'a') 'b'
False
Prelude> (\x -> x=='a') 'b'
False
```

- les différents types (parenthésage implicite à droite)

```
(==) :: Char -> Char -> Bool
(== 'a') :: Char -> Bool
(== 'a') 'b' :: Bool
```

Rappel : types polymorphes



- Une expression est polymorphe quand elle peut servir sans modifications de définition dans des contextes différents
- Rendre une fonction plus générale pour une application sur différentes données

- Une **variable de type** identifie un type quelconque (ici **a**)

- Un type est **polymorphe** comporte une variable de type

```
1 f :: a -> a
2 g :: a -> b
```

Classes Types



- Fonction plusUnEntiers sur des entiers

```
plusUnEntiers :: Int -> Int
plusUnEntiers a = a + 1
```

- Fonction plusUnFlottants sur des flottants

```
plusUnFlottants :: Float -> Float
plusUnFlottants a = a + 1
```

- Fonction plusUn avec type polymorphe

```
plusUn :: a -> a
plusUn a = a + 1
```

Classes de types



- Une **classe de types** définit un ensemble de fonctions (ou opérateurs) partagées par plusieurs types
- Pour qu'un type appartienne à une classe de types, il doit implémenter les méthodes de cette classe de types.

```
class Num a where
  (+) :: a -> a -> a
  (-) :: a -> a -> a
  (*) :: a -> a -> a
  negate :: a -> a
  abs :: a -> a
  signum :: a -> a
  fromInteger :: Integer -> a
```

- Pour qu'un type appartienne à la classe de types **Num**, il doit implémenter ses opérateurs : +, -, *, etc.

- Une fonction qui utilise + doit se limiter en ne prenant que des membres de la classe de type Num.
➡ Sinon, il ne compilera pas.

Classes de types



- Fonction plusUn avec type polymorphe

```
plusUn :: a -> a
plusUn a = a + 1
```

- Pour qu'elle puisse être compilée, on doit spécifier la classe de types pour a

```
plusUn :: (Num a) => a -> a
plusUn a = a + 1
```

Classes de types



classe	fonctions
Eq	== /=
Ord	< <= > >= min max
Num	+ - * negate abs signum
Enum	succ pred
Fractional	/ recip
Integral	div mod quot rem

